

JAVASCRIPT AVANZADO

Prototipos, memoria, npm, transpiladores y RxJS

Apuntes revisados y ampliados

Documento elaborado a partir del fastbook original de Robert Da Corte

Contenido

1. Prototipos y herencia prototipal	3
1.1. Funciones constructoras y clases	3
2. Memoria y optimización de rendimiento	4
3. Npm	4
4. Transpiladores	4
5. RxJS	5
6. Qué hemos aprendido	6

1. Prototipos y herencia prototipal

A diferencia de otros lenguajes, que reparten sus objetos a partir de clases, JavaScript utiliza la **herencia basada en prototipos**: cada objeto puede heredar propiedades y métodos directamente de otro objeto, sin necesidad de una plantilla de clase intermedia.

Casi cualquier objeto de JavaScript guarda internamente una referencia a otro objeto, su **prototipo**, a través de la propiedad interna `[[Prototype]]`. Cuando se intenta acceder a una propiedad que no existe en el objeto, el motor busca en su prototipo, y si tampoco está ahí, sigue subiendo por la cadena hasta encontrarla o hasta llegar al final, momento en el que devuelve undefined.

En el extremo de esa cadena está **Object**: casi todos los objetos de JavaScript heredan, directa o indirectamente, de `Object.prototype`, que aporta métodos como `hasOwnProperty()`, `isPrototypeOf()` o `toString()`.

1.1. Funciones constructoras y clases

Una **función constructora** es, en el fondo, una plantilla para crear objetos: define qué propiedades y métodos tendrán las instancias que se creen a partir de ella con la palabra clave `new`.

```
function Car(brand, model) {  
  this.brand = brand;  
  this.model = model;  
}  
const car1 = new Car('Toyota', 'Corolla');  
const car2 = new Car('Ford', 'Focus');
```

La palabra reservada **class**, introducida en ES6, no es más que **sugar syntax** (azúcar sintáctico) sobre este mismo mecanismo: por debajo sigue tratándose de herencia prototipal, pero con una sintaxis mucho más cercana a la de otros lenguajes orientados a objetos.

```
class Car {  
  constructor(brand, model) {  
    this.brand = brand;  
    this.model = model;  
  }  
}  
class Toyota extends Car {  
  constructor(model) {  
    super('Toyota', model); // llama al constructor de Car  
  }  
}
```

★ Lo más importante

- ▶ JavaScript hereda a través de objetos (prototipos), no de clases: class es sintaxis sobre ese mismo mecanismo.
- ▶ extends crea una clase hija; super() invoca al constructor de la clase padre.
- ▶ Object.prototype es el final de la cadena de prototipos para casi cualquier objeto.

2. Memoria y optimización de rendimiento

JavaScript incorpora un **garbage collector** (recolector de basura) que libera automáticamente la memoria que ya no está en uso, mediante un proceso llamado **mark-and-sweep**: primero marca qué variables siguen en uso y después "barre" las que no lo están.

El problema aparece cuando, por un descuido en el código, algo que ya no se necesita sigue teniendo una referencia activa en alguna parte: eso es un **memory leak** (fuga de memoria), y con el tiempo puede degradar el rendimiento de la aplicación o incluso bloquearla.

Las causas más habituales son bien conocidas: variables globales que nunca se liberan, event listeners que se añaden pero nunca se eliminan, referencias circulares entre objetos, closures mal gestionados que retienen más de lo necesario, y temporizadores (setTimeout/setInterval) que no se limpian con clearTimeout/clearInterval.

Las DevTools de Chrome (pestaña *Memory*), el flag `--inspect` de Node.js y herramientas como Lighthouse permiten tomar instantáneas de memoria y detectar este tipo de fugas antes de que lleguen a producción.

★ Lo más importante

- ▶ El garbage collector libera memoria automáticamente, pero no puede evitar todos los memory leaks.
- ▶ Vigila especialmente listeners, timers, referencias circulares y variables globales.
- ▶ Usa las DevTools de Chrome o Lighthouse para detectar fugas antes de producción.

3. Npm

Npm es el gestor de paquetes que Node.js incorpora de serie, y su aparición cambió por completo el flujo de trabajo frontend: da acceso inmediato a bibliotecas (React, Vue, Lodash...), herramientas (Babel, Webpack, ESLint...) y componentes reutilizables publicados por toda la comunidad.

```
npm init // asistente interactivo para crear package.json
npm init -y // crea package.json con valores por defecto
npm install lodash // instala una dependencia y la añade a package.json
```

El archivo **package.json** resultante concentra la configuración esencial del proyecto: nombre, versión, dependencias y scripts. Al instalar una dependencia, npm crea además la carpeta **node_modules**, donde se guardan tanto el paquete instalado como todas sus propias dependencias.

4. Transpiladores

Cuando se quiere usar sintaxis moderna de JavaScript sin renunciar a la compatibilidad con navegadores más antiguos, entra en juego un **transpilador**: una herramienta que convierte código de un lenguaje o sintaxis a otro, en este caso de ES6+ a una versión como ES5, ampliamente soportada.

Babel es, con diferencia, el transpilador más popular del ecosistema JavaScript. Su configuración básica requiere tres paquetes: `@babel/core` (el núcleo), `@babel/cli` (para usarlo desde la terminal) y `@babel/preset-env` (el preset que decide automáticamente qué transformaciones aplicar según los navegadores objetivo).

```
npm install --save-dev @babel/core @babel/cli @babel/preset-env

// .babelrc
{ "presets": ["@babel/preset-env"] }

// package.json
"scripts": { "build": "babel src -d dist" }
```

★ Lo más importante

- ▶ Un transpilador convierte JavaScript moderno a una versión compatible con más navegadores.
- ▶ Babel se configura con `@babel/preset-env` y un archivo `.babelrc`.

5. RxJS

RxJS es una biblioteca de programación reactiva basada en **observables**, muy extendida en el ecosistema Angular aunque perfectamente utilizable en cualquier proyecto JavaScript. Su gran ventaja es facilitar la composición de código asíncrono o basado en eventos.

Un **Observable** representa una colección de valores futuros: se puede pensar como una promesa capaz de emitir varios valores a lo largo del tiempo, no solo uno. Un **Observer** es el conjunto de callbacks (`next`, `error`, `complete`) que sabe qué hacer con esos valores, y la **Subscription** es lo que se obtiene al suscribir un observer a un observable.

```
const numeros$ = new Observable(subscriber => {
  subscriber.next(1);
  subscriber.next(2);
  subscriber.complete();
});

const observer = { next: valor => console.log(valor) };
const subscription = numeros$.subscribe(observer);

// Cuando ya no se necesite:
subscription.unsubscribe();
```

Los **Operators** (como `map`, `filter`, `merge` o `concat`) permiten transformar y combinar observables de forma funcional. Un **Subject** es un caso especial: se comporta a la vez como observable y como observer, y a diferencia de un observable normal (unicast), es **multicast**, es decir, todos sus suscriptores comparten la misma emisión de

valores.

★ Regla clave

- Cierra siempre las suscripciones que ya no necesites con `unsubscribe()`; de lo contrario, RxJS puede convertirse en una fuente más de memory leaks.

6. Qué hemos aprendido

Este recorrido por JavaScript avanzado ha cubierto la herencia prototipal como base real de las funciones constructoras y las clases, la gestión de memoria y cómo evitar sus fugas más comunes, el papel de npm como gestor de dependencias, Babel como puente hacia la compatibilidad entre navegadores y RxJS como forma de domar el código asíncrono mediante observables.

Son piezas que rara vez se usan de forma aislada: cualquier proyecto frontend moderno combina dependencias gestionadas con npm, código transpilado con Babel y, en aplicaciones con mucho flujo de datos asíncrono, algo parecido a RxJS por debajo.

¡Enhorabuena! Documento completado